Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №6
з дисципліни «Технології та методика програмування вбудованих систем»

«Моделювання мікроконтролерної мережі 1-Wire»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
ст. викладач, к.т.н Орленко Сергій Петрович

Дата здачі: 04.04.2026

КИЇВ – 2026

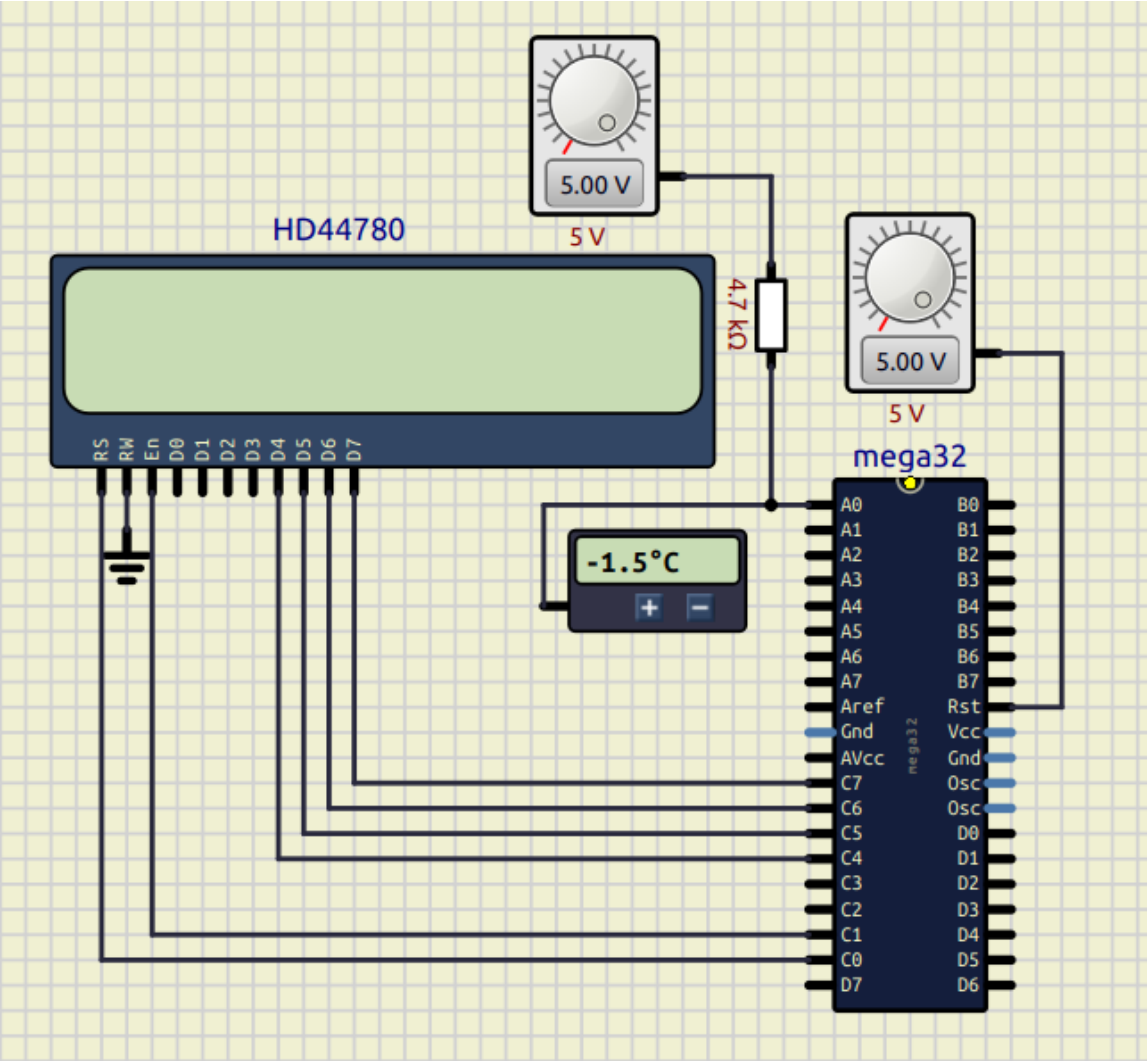
Тема. Моделювання мікроконтролерної мережі 1-Wire з використанням мікроконтролера сім'ї AVR.

Мета. Користуючись пакетом SimulIDE дослідити роботу мікроконтролерної мережі 1-Wire.

Завдання. Створити модель мікроконтролерної мережі в пакеті SimulIDE. Розробити схему алгоритму роботи мережі та робочу програму. Створити hex-файл та підключити його до мікроконтролера. Запустити модель та виконати її дослідження згідно методичних вказівок. Зробити відповідні висновки.

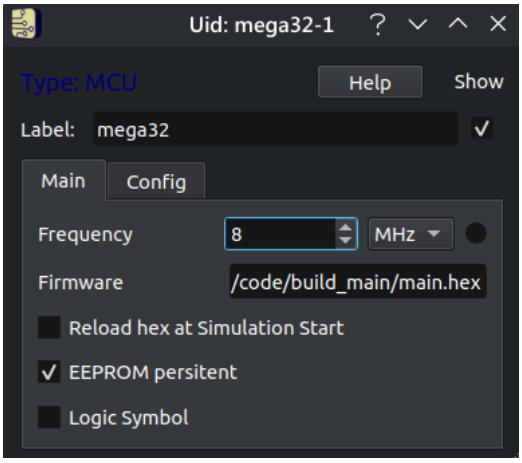
Хід роботи.

Виконання шостої лабораторної роботи було присвячено моделюванню мікроконтролерної мережі на базі інтерфейсу 1-Wire з використанням мікроконтролера сім'ї AVR.



Як альтернативу запропонованому в методичних вказівках середовищу Proteus 8.6, роботу було виконано у симуляторі SimulIDE. Основою апаратної ча-

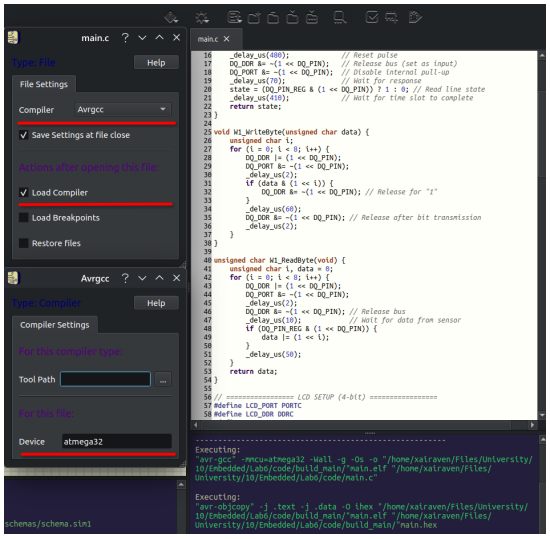
стини схеми став мікроконтролер ATmega32, тактову частоту якого було налаштовано на 8 МГц для забезпечення точних часових затримок, критично важливих для протоколу.



Для відображення результатів вимірювань до порту C мікроконтролера було підключено символьний рідкокристалічний дисплей HD44780 у чотирибітному режимі керування.

Головним сенсорним елементом системи став цифровий датчик температури DS18B20, який взаємодіє з мікроконтролером по єдиній лінії даних DQ, підключеній до піна PA0. Відповідно до специфікації однопровідного інтерфейсу 1-Wire, лінія зв’язку повинна знаходитися у стані логічної одиниці в моменти спокою. Для реалізації цієї вимоги інформаційний пін було підтягнуто до джерела живлення з напругою 5 Вольт через зовнішній резистор номіналом 4.7 кОм. Завдяки цьому забезпечується коректне формування тайм-слотів при двосторонньому обміні даними.

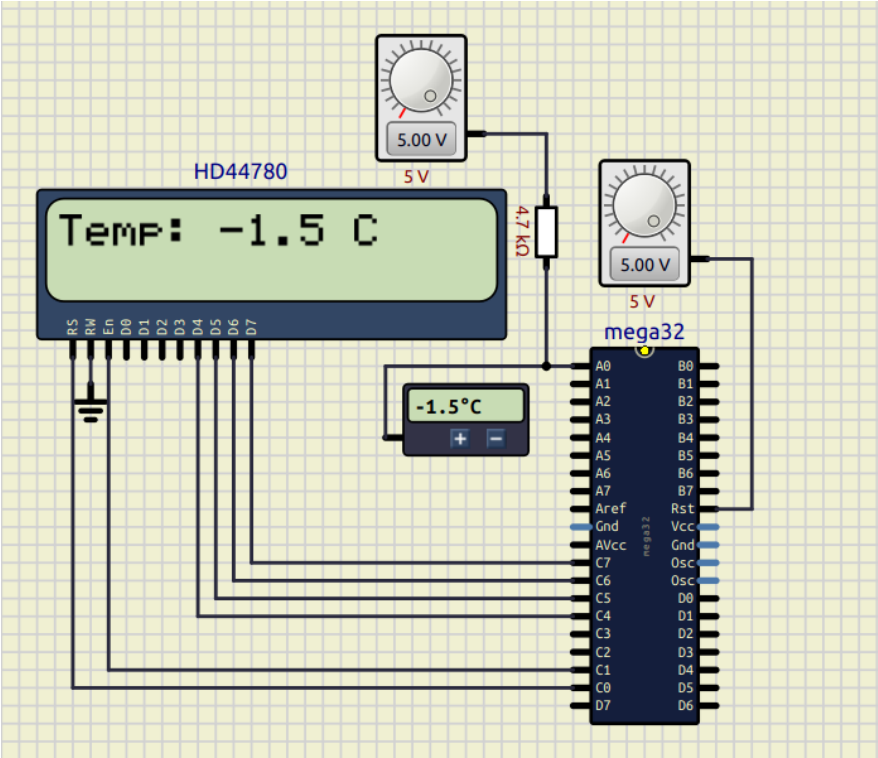
Програмну частину моделі було розроблено мовою програмування C та скомпільовано за допомогою набору інструментів avr-gcc.



Оскільки в симуляторі відсутні готові високорівневі бібліотеки, алгоритм включає самостійну реалізацію низькорівневих функцій протоколу 1-Wire. Програма генерує імпульс скидання тривалістю понад 480 мікросекунд, фіксує імпульс присутності від датчика та керує тривалістю мікросекундних тайм-слотів для побітового запису і читання інформації. Окремо було реалізовано програмний драйвер для ініціалізації та посимвольного виведення даних на дисплей HD44780.

Основний цикл програми починається з відправлення команди пропуску ідентифікації ROM (0xCC), оскільки на шині знаходиться лише один пристрій, після чого подається команда запуску температурного перетворення (0x44). Після затримки, необхідної для оцифрування температури, мікроконтролер відправляє команду читання внутрішньої пам'яті (0xBE) та послідовно зчитує два байти результату. Зчитані байти об'єднуються у 16-бітну змінну для подальшої математичної обробки. Згідно з алгоритмом, наведеним у методичних вказівках, програма перевіряє найстарший біт на наявність знаку мінус. Якщо температура від'ємна, значення інвертується шляхом доповнення до двох, а на дисплей виводиться символ мінуса. Ціла частина температури виділяється шляхом побітового зсуву вправо на чотири розряди, а дробова розраховується з молодших розрядів. У результаті моделювання на екрані дисплея було отримано стабільне відображення температури, яке миттєво і точно реагувало на зміну параметрів датчика як у додатному, так і у від'ємному діапазонах значень.

Результат:



Висновок. Під час виконання шостої лабораторної роботи було успішно досліджено принципи побудови мікроконтролерних мереж на базі однопровідного інтерфейсу 1-Wire. Практично реалізовано діючу модель системи вимірювання температури в середовищі SimulIDE, що стало повноцінною альтернативою використанню Proteus. У ході роботи було розроблено апаратну частину з мікроконтролером ATmega32, цифровим термометром DS18B20 та символьним дисплеєм HD44780, а також налаштовано обов'язкову для 1-Wire підтяжку лінії даних. Програмна реалізація мовою C дозволила на низькому рівні опанувати генерацію жорстких часових затримок (тайм-слотів) для ініціалізації шини, побітового читання та запису даних. Крім того, було успішно імплементовано алгоритм математичної обробки сирих байтів з урахуванням перевірки старшого біта для коректного відображення як додатних, так і від'ємних значень температури. Робота підтвердила надійність та зручність інтерфейсу 1-Wire для побудови розподілених сенсорних систем із мінімальною кількістю з'єднувальних ліній.

Контрольні запитання.

1. **Ким та коли було розроблено інтерфейс 1-WIRE?**
   Однопровідний інтерфейс 1-Wire був розроблений фірмою Dallas Semiconductor (яка нині є частиною Maxim Integrated) у кінці 1980-х років.
2. **Де може застосовуватись інтерфейс 1-WIRE?**
   Інтерфейс широко застосовується в приладах у спеціальних корпусах MicroCAN (технологія iButton) для вирішення проблем ідентифікації, для програмування вбудованої пам'яті інтегральних компонентів, в системах захисту доступу до ресурсів електронної апаратури, а також у системах автоматизації.
3. **Назвіть основні переваги інтерфейсу 1-WIRE.**
   Основні переваги включають мінімізацію кількості з'єднувальних дротів (потрібна лише одна лінія даних та земля), можливість живлення пристроїв безпосередньо від лінії даних (режим паразитного живлення), наявність унікального зашитого 64-бітного ROM-коду в кожному пристрої та легкість масштабування мережі.
4. **Назвіть основні характеристики інтерфейсу 1-WIRE.**
   Це напівдуплексний, двонаправлений та асинхронний інтерфейс із топологією «спільна шина» (Master-Slave). Передача здійснюється молодшим бітом вперед, а стандартна швидкість обміну даними зазвичай становить 16.3

кбіт/с (з можливістю розгону в режимі Overdrive до 142 кбіт/с).

5. **Наведіть та опишіть схему фізичної реалізації інтерфейсу 1-WIRE.**
   Фізична схема складається з одного ведучого пристрою (Master) та одного або кількох ведених (Slave), підключених паралельно до єдиної інформаційної лінії (DQ). Лінія даних є відкритою (open-drain) і обов'язково підтягується до напруги живлення через зовнішній резистор, типовий номінал якого складає 4.7 кОм.
6. **Опишіть основні правила передачі даних в мережі 1-WIRE.**
   Універсальним правилом є те, що всі транзакції ініціює Master. Сеанс починається з імпульсу скидання (ведучий притискає лінію до нуля на час більше 480 мкс), після чого відпускає її та чекає на імпульс присутності (Presence) від ведених пристроїв. Передача бітів відбувається у строгих тайм-слотах, де кожен слот починається з короткочасного притискання шини до нуля ведучим пристроєм.
7. **Назвіть обмеження застосування 1-WIRE-мереж.**
   До основних обмежень належать відносно низька швидкість передачі даних порівняно з іншими шинами, висока чутливість до паразитних ємностей довгого кабелю, а також жорсткі вимоги до мікросекундних часових затримок, що може блокувати роботу основного процесора мікроконтролера.
8. **Наведіть та опишіть структуру системи для вимірювання температури в приміщенні з використанням давача 1-WIRE.**
   Структура складається з керуючого мікроконтролера (Master), який опитує шину 1-Wire, підтягуючого резистора на лінії даних, самого давача температури DS18B20 (Slave), який підключено до шини, та пристрою відображення (LCD дисплея) для виведення результатів користувачеві.
9. **Дайте загальну характеристику датчику температури DS18B20.**
   DS18B20 – це цифровий термометр, що взаємодіє по шині 1-Wire та забезпечує вимірювання температури в діапазоні від -55°C до +125°C. Він має програмовану користувачем роздільну здатність від 9 до 12 біт та може отримувати живлення безпосередньо з лінії даних.
10. **Назвіть та опишіть команди мікроконтролера для керування мережею 1-WIRE.**
    Команди поділяються на ROM-команди для ідентифікації та функціональні. Основними є: Skip ROM (0xCC) – пропуск ідентифікації, якщо пристрій на шині один; Convert T (0x44) – запуск вимірювання температури; Read Scratchpad (0xBE) – читання внутрішньої пам'яті давача для отримання ре-

зультату.

11. **Наведіть та опишіть модель мережі 1-WIRE в SimulIDE.**

    Модель побудована на базі мікроконтролера ATmega32, що тактується на 8 МГц. До піна PA0 підключено лінію даних датчика DS18B20 та резистор підтяжки на 4.7 кОм, з'єднаний з джерелом 5 В. Результати виводяться на символьний екран HD44780, який підключений до порту C мікроконтролера у 4-бітному режимі.

12. **Наведіть та опишіть схему алгоритму роботи моделі мережі 1-WIRE в SimulIDE.**

    Алгоритм складається з ініціалізації LCD та входження у нескінченний цикл. У циклі формується імпульс скидання, після якого мікроконтролер надсилає команди Skip ROM (0xCC) і Convert T (0x44). Після затримки на вимірювання алгоритм повторює імпульс скидання, надсилає Skip ROM (0xCC) і Read Scratchpad (0xBE). Далі зчитуються два байти даних, відбувається перевірка знаку (перетворення додаткового коду для від'ємних значень), розрахунок цілої та дробової частин і виведення тексту на дисплей.

13. **Наведіть та опишіть робочу програму моделювання мережі 1-WIRE в SimulIDE.**

    Програма реалізована мовою C і складається з трьох основних блоків. Перший блок містить низькорівневі функції `W1_Init`, `W1_WriteByte` та `W1_ReadByte`, що реалізують протокол 1-Wire за допомогою точних функцій затримки `_delay_us`. Другий блок містить функції ініціалізації та посимвольного запису даних у контролер дисплея HD44780. Третій блок – це функція `main`, що циклічно опитує давач, виконує математичну обробку зчитаних байтів із виокремленням цілої частини (через зсув » `4`) та знаку, форматує рядок та виводить його на екран.

Лістинг.

main.c

```c
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h> // For sprintf function

// ================= 1-WIRE SETUP =================
#define DQ_PIN PA0
#define DQ_PORT PORTA
#define DQ_DDR DDRA
#define DQ_PIN_REG PINA

unsigned char W1_Init(void) {
    unsigned char state;
    DQ_DDR |= (1 << DQ_PIN);    // Set pin as output
    DQ_PORT &= ~(1 << DQ_PIN);  // Pull bus low
    _delay_us(480);             // Reset pulse
    DQ_DDR &= ~(1 << DQ_PIN);   // Release bus (set as input)
    DQ_PORT &= ~(1 << DQ_PIN);  // Disable internal pull-up
    _delay_us(70);              // Wait for response
    state = (DQ_PIN_REG & (1 << DQ_PIN)) ? 1 : 0; // Read line state
    _delay_us(410);             // Wait for time slot to complete
    return state;
}

void W1_WriteByte(unsigned char data) {
    unsigned char i;
    for (i = 0; i < 8; i++) {
        DQ_DDR |= (1 << DQ_PIN);
        DQ_PORT &= ~(1 << DQ_PIN);
        _delay_us(2);
        if (data & (1 << i)) {
            DQ_DDR &= ~(1 << DQ_PIN); // Release for "1"
        }
        _delay_us(60);
        DQ_DDR &= ~(1 << DQ_PIN); // Release after bit transmission
        _delay_us(2);
    }
}

unsigned char W1_ReadByte(void) {
    unsigned char i, data = 0;
    for (i = 0; i < 8; i++) {
        DQ_DDR |= (1 << DQ_PIN);
        DQ_PORT &= ~(1 << DQ_PIN);
        _delay_us(2);
        DQ_DDR &= ~(1 << DQ_PIN); // Release bus
        _delay_us(10);            // Wait for data from sensor
        if (DQ_PIN_REG & (1 << DQ_PIN)) {
            data |= (1 << i);
        }
        _delay_us(50);
    }
    return data;
}

// ================= LCD SETUP (4-bit) =================
#define LCD_PORT PORTC
#define LCD_DDR DDRC
```

```c
#define RS PC0
#define EN PC1

void LCD_Pulse(void) {
    LCD_PORT |= (1 << EN);
    _delay_us(1);
    LCD_PORT &= ~(1 << EN);
    _delay_us(50);
}

void LCD_Send(unsigned char value, unsigned char mode) {
    if (mode) LCD_PORT |= (1 << RS); // Data mode
    else LCD_PORT &= ~(1 << RS);     // Command mode

    // Send high 4 bits
    LCD_PORT = (LCD_PORT & 0x0F) | (value & 0xF0);
    LCD_Pulse();

    // Send low 4 bits
    LCD_PORT = (LCD_PORT & 0x0F) | ((value << 4) & 0xF0);
    LCD_Pulse();
    _delay_ms(2);
}

void LCD_Init(void) {
    LCD_DDR = 0xFF; // Port C as output
    _delay_ms(20);

    // HD44780 initialization procedure
    LCD_PORT = (LCD_PORT & 0x0F) | 0x30;
    LCD_Pulse(); _delay_ms(5);
    LCD_Pulse(); _delay_us(150);
    LCD_Pulse(); _delay_us(150);

    LCD_PORT = (LCD_PORT & 0x0F) | 0x20; // Switch to 4-bit mode
    LCD_Pulse(); _delay_ms(2);

    LCD_Send(0x28, 0); // 4-bit, 2 lines, 5x8 font
    LCD_Send(0x0C, 0); // Display ON, Cursor OFF
    LCD_Send(0x01, 0); // Clear display
    _delay_ms(2);
}

void LCD_String(char* str) {
    while (*str) {
        LCD_Send(*str++, 1);
    }
}

// ================= MAIN PROGRAM =================
int main(void) {
    unsigned char t_low, t_high;
    int temp16;
    unsigned char temp_int;
    unsigned char frac_raw;
    unsigned int frac_decimal;
    char buffer[16];

    LCD_Init();
    LCD_String("1-Wire System");
    LCD_Send(0xC0, 0); // Move to second line
    LCD_String("Loading...");
    _delay_ms(1000);
```

```c

    while (1) {
        LCD_Send(0x01, 0); // Clear display

        // If sensor is present (responded with 0)
        if (W1_Init() == 0) {
            // 1. Start conversion command
            W1_WriteByte(0xCC); // Skip ROM (only one device on bus)
            W1_WriteByte(0x44); // Start conversion (Convert T)

            _delay_ms(750);     // DS18B20 requires up to 750ms for 12-bit conversion

            // 2. Read data command
            W1_Init();
            W1_WriteByte(0xCC); // Skip ROM
            W1_WriteByte(0xBE); // Read Scratchpad

            t_low = W1_ReadByte();  // Low byte
            t_high = W1_ReadByte(); // High byte

            temp16 = (t_high << 8) | t_low; // Combine into 16 bits

            LCD_String("Temp: ");

            // Sign check
            if ((temp16 & 0x8000) == 0) {
                LCD_Send('+', 1);
            } else {
                LCD_Send('-', 1);
                temp16 = ~temp16 + 1; // Invert value for negative temperature
            }

            // Shift right by 4 to get integer part
            temp_int = (unsigned char)(temp16 >> 4);

            // Fractional part
            frac_raw = temp16 & 0x0F;
            frac_decimal = (frac_raw * 10) / 16;

            // Display the result
            sprintf(buffer, "%d.%d C", temp_int, frac_decimal);
            LCD_String(buffer);
        } else {
            LCD_String("Sensor Error!");
        }

        _delay_ms(500); // Delay before next polling
    }
    return 0;
}
```